AI駆動開発 大阪支部 Jev特別会 ・ 鈴山英寿（RAYVEN）

Jev の中身は 50行で再現できる

小さな LLM で Jev を自作し、本物と同じ30問で比べたら、差は「自信の正しさ」にあった

▶ PRESS START

じこしょうかい – ステータス

鈴山 英寿（すずやま ひでひさ）／ RAYVEN CTO

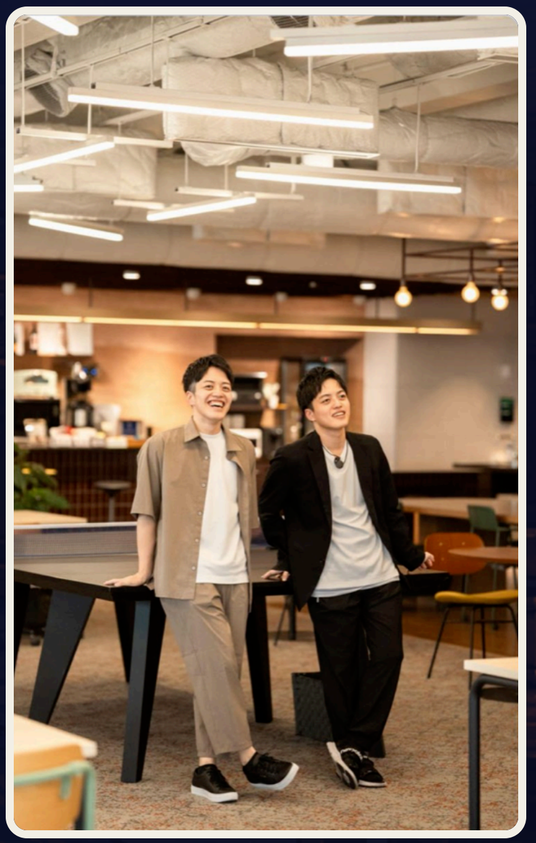
これまでの ぼうけん

学生時代　フリーランスエンジニア（スクールのメンター、受託開発）

2023.04　ヤフー入社。CTO 直下の SWAT チームで技術課題を解決

2023.10　LINEヤフー統合1年目に PayPayカードへ約1年出向

2024.07　RAYVEN を共同創業。開発から採用・営業まで



しゅみ　サウナ・テニス　　いま　VIVANT を一気見中

いまの クエスト　AI が働ける組織づくり。社内で Jev も使い始めた

ギルド しょうかい – 株式会社RAYVEN

RAYVEN

AIエージェントにも、社員証を。

人は入社すると社員証をもらい、入れる部屋と使えるシステムが決まる。AI にも同じしくみを

AI の社員証を発行する

Tumiki

AI エージェントに専用の ID を渡し、触れてよいデータとツールを決める。どの AI が、誰の指示で、何をしたかを記録する

AI の配属先を見つける

WorkLens AI

PC 業務の記録から、AI に任せられる業務と減らせる時間を見つける。解析方法の選択に Jev を使用

AI の導入を決めるところから、安全に働かせるところまで。社内でも自分たちで使い、手続きと記録は AI、判断は人で回している

3 「AIエージェントにも、社員証を。」は CEATEC 2026 出展時のコンセプト

Jev は「選ぶだけ」の AI

文章を生成せず、選択肢ごとの確率（＝自信）を返す。公称で 193.6倍速く、444.6倍安い（TypeSafe AI）

この中身、自分で作れる？

LLM は「次の1語」を確率で予想しているだけ

入力した文章

日本の首都は

次に来る1語は？

LLM が出す「次の1語」の確率の表（全約15万語）

…… 残り約15万語にも、ごく小さな確率がつく

1語＝トークン（LLM が扱う文字のかたまり）。「東京」も「東」と「京」の2つに分かれる

チャットの返事は、この予想のくり返しでできている

1回目　日本の首都は東　← 69.5% でいちばん高い1語を書く

2回目　日本の首都は東京　← 91.5%

3回目　日本の首都は東京です　← 36.3%

4回目　日本の首都は東京です。　← 91.7%

ChatGPT や Claude の返事も、これを数百〜数千回くり返したもの

「考える」モデル（thinking）は、答える前に考える文章も同じ方法で書く。その分だけ遅い（実測：4B で 0.6秒 → 21.7秒）

6　Qwen3 4B の実測値。毎回いちばん確率の高い1語を選んだ場合

書かせずに、最初の1語の確率だけを覗く

いつものチャット

予想 → 書く → 予想 → 書く → ……

数百〜数千回くり返して、文章で答える

自信の度合いは、文章からは分からない

今回の自作（Jev 風）

予想を1回だけ。何も書かせない

確率の表から A / B / C の行だけ読む

→ 答えと「自信（確信度）」が数字で出る

→ 自信が低い答えだけ、人に確認を回せる

たとえるなら：相手が口を開く直前に「A と言いそう 7%、B と言いそう 87%」と心の中を覗いて、そこで止める

7 「考える」機能はオフにする。オンだと最初の1語が「考え始めの合図」になり、A/B/C が読めない

選択肢に記号を振り、LLM に1回だけ読ませる

LLM に渡すプロンプト（実物）

```
# 状況
お客様：昨日から本番環境にログインできません。今日の15時の商談デモで使う予定です。

# 質問
深刻度は？

# 選択肢
A. 低　B. 中　C. 高

最も適切な選択肢の記号を1文字だけ答えてください。
```

1. 選択肢に記号を振る

A / B / C は、どれも LLM にとって1トークン

2. 文章は生成させない

プロンプトを1回通す（forward）だけ。答えを書かせない

3. 次の1トークンを予想させる

LLM は「次に来る語」の確率を全語彙について出す

モデルは Qwen3 1.7B / 4B（4bit 量子化）を MLX で M4 Mac 上で実行。thinking は無効化

1回の予想で出てくるのは、約15万個の数字の列

モデルの出力（生の形）

(1, 97, 151936)

97 ＝ 入力したトークンの数

151,936 ＝ 語彙の数。1語ごとにスコアが1つ

使うのは最後の位置の1行だけ（＝次の1語の予想）

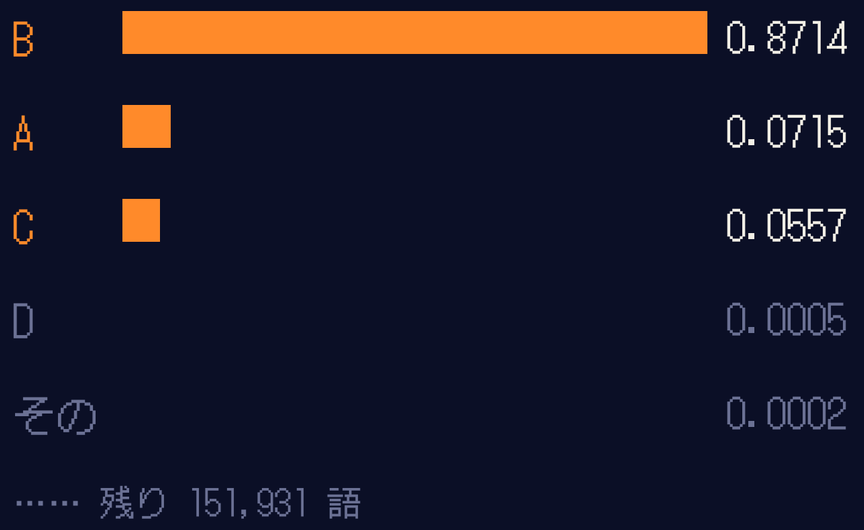
番号	1語	スコア	確率
33	B	30.12	87.14%
32	A	27.62	7.15%
34	C	27.38	5.57%
35	D	22.62	0.05%
57326	その	21.75	0.02%
……	……	……	……
50000	RESET	-3.58	ほぼ 0%

スコア（ロジット）を softmax で確率に直すと、全15万語で合計100%。スコアの差が 2.5 開くと確率は約12倍

9　自作 1.7B に深刻度の質問（A. 低 / B. 中 / C. 高）を入れたときの実際の出力。モデルの中では1語ずつ番号で扱われる

次トークンの確率から A/B/C だけ取り出す

次トークンの確率（全 151,936 語の上位）



A/B/C だけ残して、合計を1にする

A. 低　0.07

B. 中　0.87　← いちばん高い

C. 高　0.06

答え：中

確信度：0.87

10　自作 1.7B の実測値。A/B/C 以外の確率は合計 0.13% で、捨てても答えはほぼ変わらない

全部で53行。確率を取り出す核心は5行だけ

言語：Python（Apple の機械学習ライブラリ MLX）　ファイル：minijev.py の核心部分

```
logits = model(mx.array(tok.encode(prompt))[None])[0, -1]

full = mx.softmax(logits.astype(mx.float32))

picked = full[mx.array(LABEL_IDS[: len(options)])]

mass = picked.sum().item()

return dict(zip(options, (picked / mass).tolist())), mass
```

① プロンプトを1回だけモデルに通し、次の1語のスコアを全15万語ぶん受け取る

② スコアを確率に直す（全部足すと100%）

③ A・B・C の行だけを取り出す

④ A・B・C の確率の合計を出す

⑥ 合計で割って100%に直し、「選択肢 → 確率」の形で返す

装備：Python ＋ MLX ＋ Qwen3 1.7B / 4B（4bit）・M4 の Mac で動く ・ネット不要　github.com/hisuzuya/minijev

自作でつまずいた点は3つ（本物なら不要）

シンキングスライム

thinking を切らないと、最初のトークンが <think> になる

ふたごヘビ

「A」と「 A」は別トークン。読みラベルは1トークン。違えると確率を取りこぼす（合算しても今回は差なし）

26択ゴーレム

A〜Z の26択まで（Jev は255択）

動いた。でも、まちがえても自信満々

入力：在庫は りんご12個・みかん9個・ぶどう15個。注文は みかん10個。

勇者は「判定」を はなった！

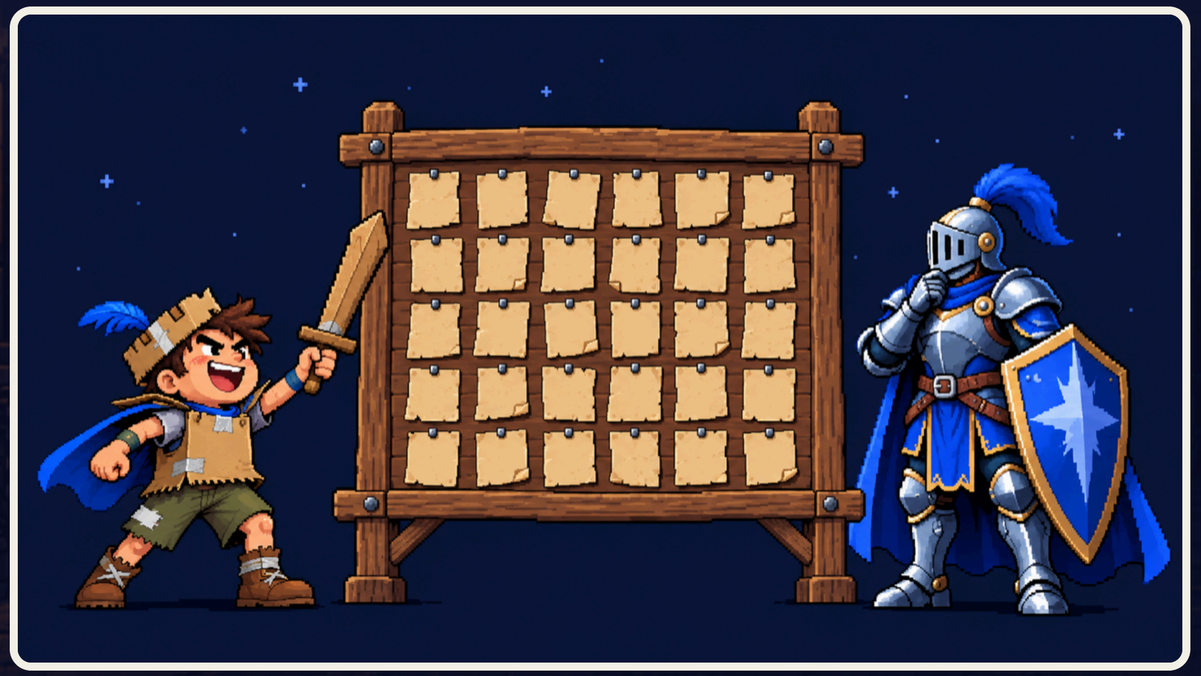
質問 …… 在庫だけで 注文に対応できる？

勇者の答え …… はい（確信度 1.00）

騎士の答え …… いいえ（確信度 0.96）

……しかし みかんは 9個しかない！ 勇者は 自信満々で まちがえた！

同じ30問を4者に解かせて比べる



クエスト（自作・正解は手で付けた）	問数
基本の分類	10
否定・複合条件	5
数値・カウント　※	5
日付・期間　※	5
多選択肢（12択）	5

パーティ：勇者 Lv1（自作 1.7B） ・ 勇者 Lv4（自作 4B） ・ 騎士（本物の Jev）
賢者（Opus 5.5 に JSON スキーマで答えの形を固定させたもの）

　※ Jev の公式資料で「苦手」とされている分野。以降の比較の主役は勇者 Lv4（自作 4B）。Lv1（1.7B）は参考

正答率より「自信ありで間違えた数」を見る

業務での使い方：確信度 0.7 以上なら自動で処理、0.7 未満なら人に回す

◎ 自動で正解

自信ありで当たった。人の手間ゼロ。多いほど良い

○ 人に回す

自信なしと正直に言った。人の手間はかかるが安全

× 自動で誤り＝事故

自信ありで外した。誰も気づかず素通りする。0件が理想

ものさし：当てずっぽうで答えると 30問中 約10.5問 正解。理想は「事故0件、人に回すのは間違える問題だけ」

15　補助指標として Brier スコア（確信度と正誤のずれの2乗平均。0 が理想）も計算した

正答率：自作 4B は本物に迫り、Opus は満点

勇者 Lv1（自作 1.7B）　17/30

勇者 Lv4（自作 4B）　LEVEL UP！　25/30

騎士（本物の Jev）　27/30

賢者（Opus 5.5 ＋ JSON 固定）　30/30

↑ 当てずっぽうなら 10.5

……もう勇者でいいのでは？

30問・各1回・自作の合成データ。否定・数値・日付は勇者 Lv4 と騎士が同点（5/5 ・ 4/5 ・ 3/5）

本物の Jev は、外した3問とも確信度 0.55 未満

勇者 Lv4

騎士

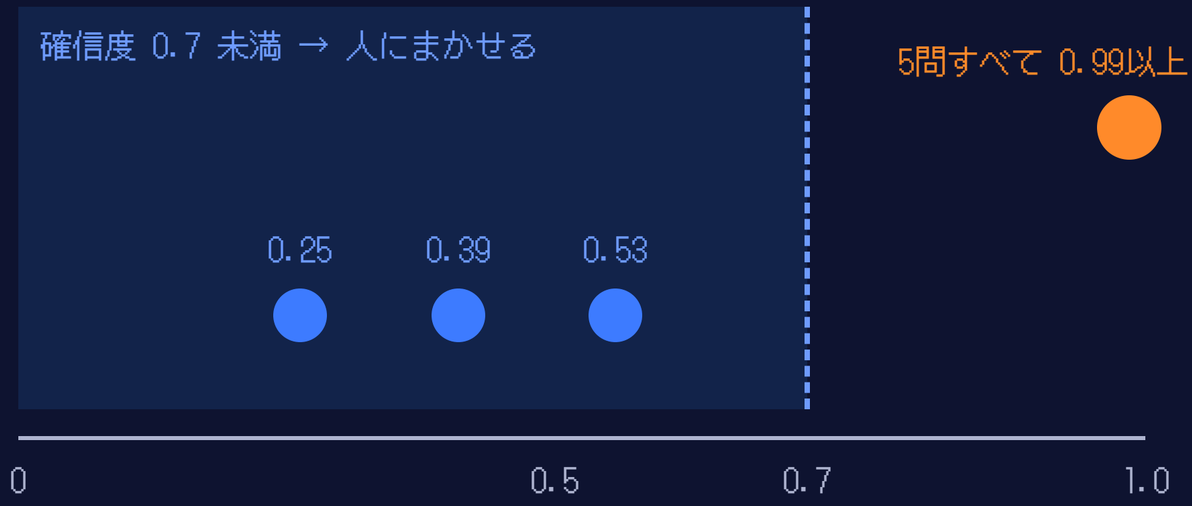
騎士は 迷ったら まず調べる。
外した3問は すべて 人に回り、事故にならない。

17　間違えた問題だけを確信度の位置に並べた。賢者（Opus）は外れ0問のため点なし

自作 4B は、外した5問とも確信度 0.99 以上

勇者 Lv4

騎士



勇者は 自信満々で ミミックに飛びこんだ。
5問とも 人に回らず、誰にも気づかれず 素通りする。

0.7 で運用すると、事故は 自作4B 5件・Jev 0件

30問の結果	正答	◎ 自動で正解	○ 人に回す	× 事故	Brier
勇者 Lv1（自作 1.7B）	17	17	0	13	0.418
勇者 Lv4（自作 4B）	25	24	1	5	0.171
騎士（本物の Jev）	27	27	3	0	0.021
賢者（Opus 5.5 ＋ JSON）	30	29	1	0	0.012

騎士は 外した3問を 全部 人に回せた。
勇者 Lv4 は 正答25問でも、5問が 誰にも気づかれず素通り。

19　確信度0.7以上を自動処理、未満を人に回したときの内訳。Brier は確信度と正誤のずれ（0 が理想）

実測：Opus＋JSON 固定も事故0。Jev は7倍速く桁違いに安い

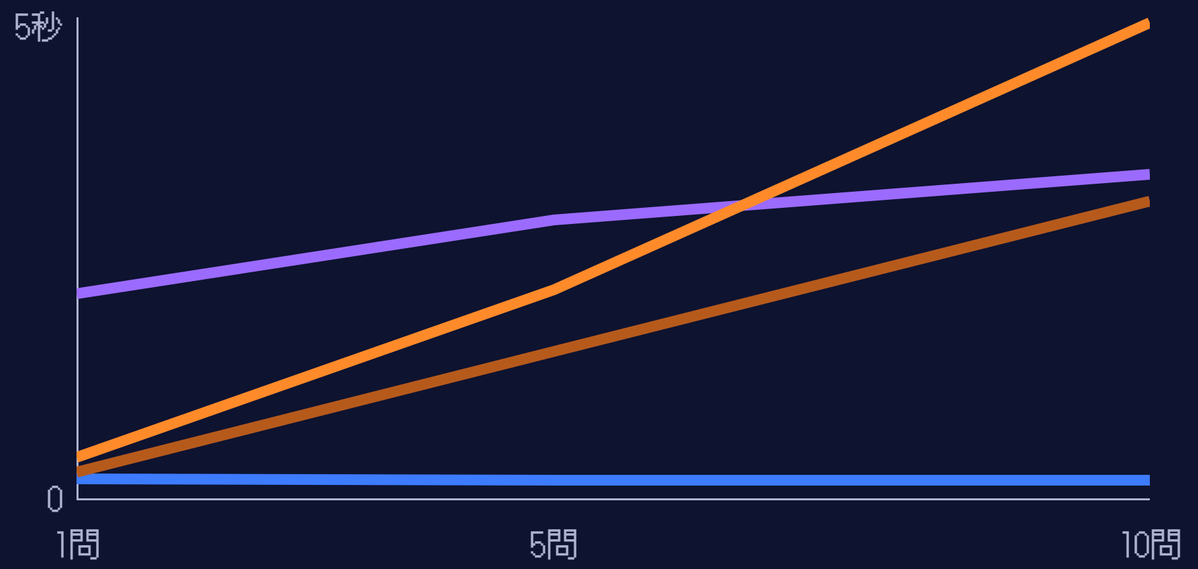
	騎士（Jev）	賢者（Opus 5.5）
正答	27 / 30	30 / 30
事故（0.7 運用）	0 件	0 件
1問の応答	0.22 秒	1.6 秒
10問まとめて	0.21 秒	3.4 秒
1問の費用（目安）	約 $0.00002	約 $0.023

賢さなら賢者。
大量の判定を速く安く回すなら騎士。

20　Opus は Claude Code CLI の --json-schema（effort low）で実行。確信度は自己申告。時間は API 時間、費用は CLI の定価見積もり。Jev の費用は公称単価（入力 $0.042/100万トークン・出力無料）× 実測381トークンの概算

第3章　たたかう ・ ROUND 3 質問を増やしたとき – 勇者は 息が きれてきた！

10問まとめて聞いても、Jev だけ 0.2秒のまま



10問のとき：騎士 0.21秒　賢者 3.4秒　勇者 Lv1 3.1秒　勇者 Lv4 4.9秒

勇者は 質問が増えるほど 息切れ。騎士は 何問でも 涼しい顔。

21　線は実際の応答時間と同じ速さで伸びる。同じ状況に質問を1・5・10問（3回の中央値）。条件は同じではない（騎士は API 往復込み、勇者は M4 ローカル、賢者は API 時間）

選択肢の順番を変えると、自作は答えがぶれる

勇者 Lv1（自作 1.7B）
9 / 17

勇者 Lv4（自作 4B）
11 / 17

騎士（本物の Jev）
16 / 17

賢者（Opus 5.5）
17 / 17

勇者は 順番を変えられるだけで、3回に1回 まどわされた。
騎士と賢者は ほぼ ぶれない。

　選択式の17問で、選択肢の順番を3通りに入れ替えても答えが変わらなかった数

自作に足りないのは自信の正しさ。Jev の強みは速さと安さ

ステータス	勇者 Lv4（自作）	騎士（Jev）	賢者（Opus）
正答率	○ 25 / 30	○ 27 / 30	◎ 30 / 30
自信の正しさ（事故）	× 5件	◎ 0件	◎ 0件
10問まとめて	× 4.9秒	◎ 0.21秒	△ 3.4秒
並び順への強さ	△ 11 / 17	◎ 16 / 17	◎ 17 / 17
1問の費用（目安）	◎ 0円（手元）	◎ 約$0.00002	△ 約$0.023
データを外に出さない	◎ 手元で完結	× API	× API

53行で再現できたのは「仕組み」まで。自信の正しさは、学習やモデル規模の差と考えられる

23　Jev の内部構造は非公開。並列評価は公式仕様、学習は RLCD と説明されている。それ以上は推測

使い分け：試作は自作、大量は Jev、難問は Opus

勇者（自作）
試作・機密データ・オフライン。確信度は閾値に使わない

騎士（本物の Jev）
大量の判定を速く安く。確信度で人に回す本番運用

賢者（Opus ＋ JSON 固定）
件数が少なく、正確さ最優先の判定。理由も欲しいとき

「答え」は 小さいモデルでも 追いつける。
「自信の正しさ」は 追いつけなかった。

注意：30問・各1回の小さな実験です

- 本編は自作の30問・各1回。100問・14B までの再検証は付録に（結論は同じ）
- 勇者は確信度の補正（温度スケーリングなど）を試していない
- 速度は条件をそろえていない（騎士はネットワーク込み）
- 賢者は Claude Code CLI 経由。確信度は自己申告、費用は定価の目安

つづく：補正の装備で、勇者はどこまで強くなるか

コード：github.com/hisuzuya/minijev

まとめ – ぼうけんの きろく

今日の持ち帰り3つ

1. 作れた

LLM は次の1語の確率を出しているだけ。書かせずに確率を読めば、53行で Jev 風の「選ぶ AI」になる

2. でも差があった

自作は 4B〜14B で正答率8割超まで来たが、外したときも確信度 0.99。自信で人に回す設計が成り立たない

3. 使い分ける

Jev の価値は「自信の正しさ」と速さ・安さ。試作は自作、大量は Jev、難問は Opus

ご清聴ありがとうございました

鈴山 英寿（RAYVEN CTO）

WE'RE HIRING – なかまを ぼしゅうちゅう！

AI が好きな人、技術登壇に興味がある人を募集しています。
AI駆動開発 大阪支部の LT 登壇・運営への参加も歓迎です

Wantedly wantedly.com/companies/company_4392135　公式サイト rayven.co.jp

Wantedly　公式サイト

RAYVEN

コードは こうかい中！

github.com/hisuzuya/minijev

コード53行・評価データ・結果・
スライド PDF をだれでも見られます
（MIT ライセンス）

付録1 – 100問で再検証

100問に増やしても、結論は同じだった

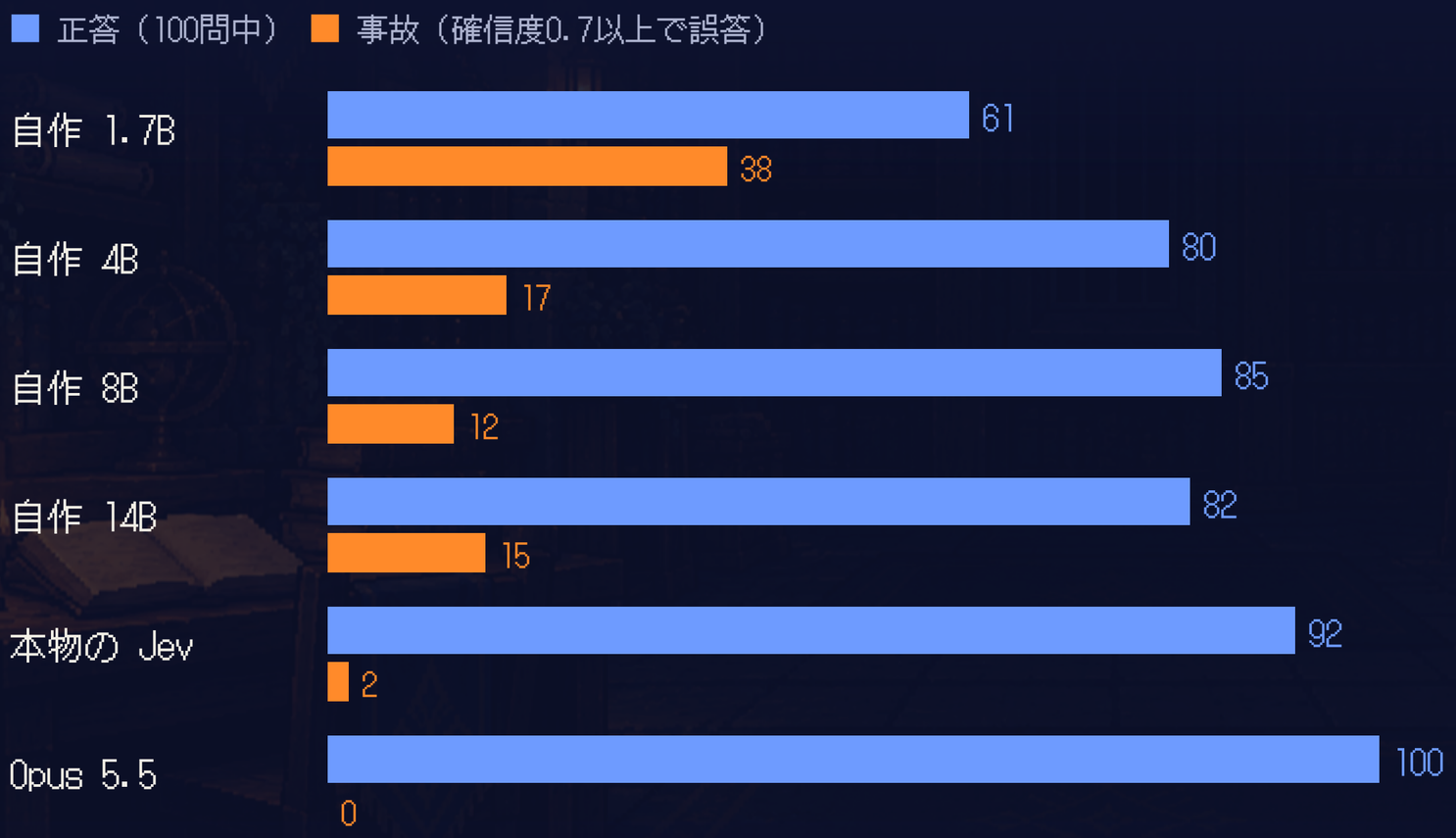
	重み	正答（95%CI）	事故	外したときの確信度	10問まとめて
自作 1.7B	0.9GB	61%（51〜70）	38	1.00	3.5秒
自作 4B	2.1GB	80%（72〜88）	17	1.00	5.8秒
自作 8B	4.3GB	85%（78〜92）	12	1.00	12.9秒
自作 14B	7.8GB	82%（74〜89）	15	1.00	16.3秒
本物の Jev	API	92%（87〜97）	2	0.61	0.17秒
Opus 5.5	API	100%	0	–	3.3秒

30問では「4B は Jev に迫る」と言えたが、100問では一段下（80% と 92%、信頼区間はほぼ重ならない）。自作の誤答は大きさに関係なく確信度 1.00

A1　事故＝確信度0.7以上で誤答（自動処理されてしまう）。でたらめに選ぶと 34.8/100。M4 Mac・4bit、2026-10-29

付録2 – 勇者を大きくしてみた

大きくしても「自信満々で外す」は治らない



正答：1.7B→4B で大きく伸び、その先は 85・82 で頭打ち。14B でも Jev に届かない

事故：4B〜14B で 12〜17件のまま。0.99以上の誤答は 14B でも13件

速さ：14B は10問で16秒。Jev の約100倍遅い

A2　自作は mlx-community/Qwen3-*-4bit。14B は 7.8GB、M4 32GB で実行

付録3 – どこで つまずくか

自作がつまずくのは「数値」と「日付」

	基本（33）	否定・複合（17）	数値（17）	日付（17）	多選択肢（16）
自作 1.7B	27	13	3	6	12
自作 4B	33	14	11	8	14
自作 8B	32	16	9	12	16
自作 14B	32	16	10	9	15
本物の Jev	33	17	13	13	16
Opus 5.5	33	17	17	17	16

基本と多選択肢は 4B 以上でほぼ満点。誤りは税込計算・合計・割合・単位換算と曜日・営業日・期限の前後に集中。1トークンで答える方式では、計算の途中を書けないため

A3　詳細は minijev の results/v2/SUMMARY.md と results/SCALE.md。数値は2026-10-29の1回分